Technische Hochschule Nürnberg

Fakultät Informatik

KI-gestützte Transformation statischer in interaktive 3D-Objekte durch Benutzerinteraktion

Bachelorarbeit im Studiengang Medieninformatik

vorgelegt von

Friedrich Allenhof

Matrikelnummer 359 4928

Erstgutachter: Prof. Dr. Uwe Wienkop

Zweitgutachter: Louis Burk

© 2026

Dieses Werk einschließlich seiner Teile ist **urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtgesetzes ist ohne Zustimmung des Autors unzulässig und strafbar. Das gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmungen sowie die Einspeicherung und Verarbeitung in elektronischen Systemen.

Abstract

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Inhaltsverzeichnis

1. Einführung **1**
1.1. Problemstellung 1
1.2. Forschungsfrage und Zielsetzung 2
1.3. Abgrenzung 3
1.4. Aufbau der Arbeit 3

2. Grundlagen **5**
2.1. Statische und interaktive 3D-Objekte 5
2.2. Transformation statischer in interaktive 3D-Objekte 6
2.3. Das Vivian Framework 6
2.3.1. Spezifikationen 7
2.4. Large Language Models und Agentische Systeme 7
2.4.1. Definition 7
2.4.2. Multimodale LLMs 8
2.4.3. Verwendung in der vorliegenden Arbeit 8

3. Stand der Forschung **11**
3.1. Artikulierung von statischen Objekte 12
3.2. Bildbasierte Eingaben 12
3.3. Einordnung der vorliegenden Arbeit 13

4. Methodik **15**
4.1. Forschungs- und Entwicklungsansatz 15
4.2. Anforderungsanalyse 15
4.2.1. Funktionale Anforderungen 15
4.2.2. Nicht-funktionale Anforderungen 16
4.2.3. Domänenspezifische Anforderungen 16
4.3. Spezifikationsgenerierung für das Vivian Framework 17
4.4. Konzeption der Pipeline 17
4.4.1. Scene Analyzer 17
4.4.2. Interaction Planner 17
4.4.3. Specification Generator 17
4.4.4. Consistency Reviewer 17
4.4.5. Validator 17

5. Implementierung **19**

6. Evaluation **21**

7. Ausblick **23**

8. Fazit **25**

A. Supplemental Information **27**

Abbildungsverzeichnis **29**

Tabellenverzeichnis **31**

Listingverzeichnis **33**

Literatur **35**

Kapitel 1.

Einführung

1.1. Problemstellung

In der Regel enthalten statische 3D-Objekte Geometrie, Materialien und Transformationen, jedoch kein Wissen darüber, welche expliziten Teile bedienbar sind, welche Funktion sie erfüllen oder wie sie auf Nutzer*innenhandlungen reagieren. Für Interaktivität sind aber genau diese Informationen von fundamentaler Bedeutung, nämlich wenn Objekte oder Teilobjekte mit Bedeutung, möglichen Handlungen und Verhalten verbunden werden [1]. Somit bestehen virtuelle, interaktive Prototypen nicht nur aus einer 3D-Geometrie, sondern besitzen zusätzlich semantisch-verhaltensbezogene Informationen.

Die Erstellung interaktiver 3D-Szenen oder die Transformation dieser in interaktive Prototypen setzt oft ein technisches Verständnis und Wissen voraus, wie etwa in 3D-Modellierung, Szenenaufbau oder Programmierung, was die Einstiegshürde für potenzielle Anwendende wie Produktentwickler*innen, Designer*innen oder Lehrende vergrößert. Diese Gruppe von Anwender*innen wird in dieser Arbeit als nicht-technische Benutzer*innen bezeichnet, also Personen ohne Kenntnisse in der Programmierung oder 3D-Modellierung. Die Fähigkeit fachliche Anforderungen und Funktionalität präzise formulieren zu können, wird dabei jedoch vorausgesetzt.

Interaktivität wird meist an konkrete Objekte oder Teilobjekte gebunden. Verfahren wie 3D-Scanning erlauben allerdings häufig nur eine monolithische Erzeugung statischer Abbildungen der Realität. Auch rekonstruierte Szenen aus Punktwolken bieten an sich keine einzeln adressierbaren Teile. Diese Abbilder sind somit überwiegend nicht semantisch segmentiert und Objekte existieren nicht als eigenständige, adressierbare Entitäten. Eine interaktive Anpassung oder Segmentierung im Nachhinein ist dann nur mit massivem manuellem Aufwand möglich.

Weiterhin muss Interaktionsverhalten in eine konsistente, maschinenlesbare Repräsentation überführt werden, da interaktive Prototypen mehr als eine rein textuelle Beschreibung des gewünschten Verhaltens benötigen. Diese Überführung ist in der Regel ebenfalls mit hohem manuellem Aufwand verbunden und dadurch auch fehleranfällig.

Außerdem fehlen in automatisierten Verfahren zur Erstellung und Bearbeitung von virtuellen Prototypen oft interne Prüf- oder Korrekturmaßnahmen, um Fehler frühzeitig und im Prozess erkennen und beheben zu können. Diese Fehler müssen dann häufig zeitaufwendig und manuell im Nachhinein behoben werden. Auch können Halluzinationen auftreten, wie etwa in 3D-LLMs, wodurch Objekte teils nicht existieren oder falsche Beziehungen zwischen Objekten bestehen [2].

Daraus lässt sich das zentrale Problem dieser Arbeit ableiten: Es fehlt ein niedrigschwelliger und dennoch kontrollierbarer Ansatz, mit dem bereits vorhandene statische 3D-Objekte in maschinenlesbare, validierbare und zustandsbasierte Prototypen überführt werden können, während die Geometrie beibehalten wird.

1.2. Forschungsfrage und Zielsetzung

Um den in 1.1 beschriebenen Problemen zu begegnen, untersucht die vorliegende Arbeit die Möglichkeit einen Ansatz zu entwickeln, welcher die automatisierte Überführung statischer 3D-Objekte in validierte, Zustands-behaftete, interaktive Prototypen ermöglicht, der insbesondere auch nicht-technischen Benutzer*innengruppen eine Nutzung ermöglicht und die Geometrie der Teilobjekte beibehält. Im Zentrum steht die folgende Forschungsfrage:

> **Lässt sich eine automatisierte Pipeline konzipieren und prototypisch umsetzen, die vorhandene statische 3D-Objekte mittels natürlicher Sprache in interaktive Prototypen überführt und dabei vorhandene Geometrien beibehält?**

Zur detaillierten Beantwortung dieser Frage werden die folgenden untergliederten Forschungsfragen definiert:

- **F1:** Kann ein Co-Creation-Workflow entworfen werden, der es auch nicht-technischen Benutzer*innengruppen ermöglicht, statische 3D-Objekte in interaktive Prototypen zu überführen?
- **F2:** Lassen sich relevante Objekte, Teilobjekte und semantische Rollen aus vorhandenen Szeneninformationen und Nutzer*innenbeschreibungen ableiten?
- **F3:** Können komplexe Interaktionslogiken aus der Kombination von bestehenden 3D-Geometrien und natürlicher Sprache extrahiert und in eine maschinenlesbare Repräsentation überführt werden?
- **F4:** Inwieweit können Validierungs- und Selbstkorrekturmechanismen bei der automatisierten Erstellung von Prototypen syntaktische, semantische und referenzielle Fehler vor der Laufzeit reduzieren?

Das Ziel dieser Arbeit ist die Konzeption und prototypische Implementierung einer KI-gestützten Pipeline, welche eine Überführung von statischen 3D-Objekten in interaktive Prototypen vornimmt und dabei natürliche Sprache entgegennimmt, um auch nicht-technischen Benutzer*innengruppen einen Zugang zu bieten. Dabei sollen die Fähigkeiten von LLMs genutzt werden und ein Co-Creation-Workflow entstehen, um einen Menschen in den Generierungsprozess einzubeziehen.

1.3. Abgrenzung

Diese Arbeit zielt nicht darauf ab, 3D-Szenendaten oder Geometrie zu generieren. Das beinhaltet komplette 3D-Szenen sowie einzelne 3D-Objekte, wie es in einigen verwandten Arbeiten der Fall ist. Es wird ausschließlich die Spezifikationsableitung bereits vorhandener virtueller Szenen betrachtet, um virtuelle Prototypen durch Verwendung des Vivian Framework zu erschaffen.

1.4. Aufbau der Arbeit

Aufbauend auf der in Kapitel 1 genannten Problemstellung werden in Kapitel 2 einige theoretische Grundlagen erläutert, wie statische und interaktive 3D-Objekte, das Vivian-Framework sowie LLMs und agentische Systeme, um ein fachliches Fundament für diese Arbeit zu bilden.

In Kapitel 3 wird der derzeitige Stand der Forschung zur Verwendung von LLMs in virtuellen 3D-Welten eingeordnet und Herausforderungen aufgezeigt, welche zur Motivation der Zielsetzung dieser Arbeit dienen. Zusätzlich wird ein Bezug zur Forschungslücke in dem Gebiet gegeben.

Kapitel 2.

Grundlagen

2.1. Statische und interaktive 3D-Objekte

Im Folgenden werden statische sowie interaktive 3D-Objekte definiert und erklärt.

Statische 3D-Objekte

In dieser Arbeit werden statische 3D-Objekte als Basisform digitaler Modelle betrachtet. Sie sind die fundamentale Basis, auf denen komplexere Verhaltensweisen aufbauen können, jedoch wohnt ihnen mit diesen Informationen noch kein Eigenleben inne. Statische 3D-Objekte lassen sich meist rein geometrisch beschreiben. Die Mesh-Geometrie besteht aus Dreiecksnetzen oder Polyeder-Darstellungen, welche die Oberfläche durch Eckpunkte (Vertices) und deren Verbindungen definieren [3]. Weiterhin besitzen sie grafische Attribute wie Materialeigenschaften, Texturen, Transparenz und Sichtbarkeit sowie eine hierarchische Anordnung im Szenengraph, welcher ihre Position, Rotation und Skalierung organisiert [4].

Interaktive 3D-Objekte

Während statische 3D-Objekte als unbelebte geometrische Körper betrachtet werden können, zeichnen sich interaktive 3D-Objekte durch Wissen über ihre Funktionalität und die Möglichkeit zur Interaktion mit ihnen aus [1]. Auf diese 3D-Objekte kann durch Eingaben (Input) eingewirkt werden und das Objekt reagiert zustands- oder verhaltensbezogen [5]. Diese Reaktion verleiht der unbelebten Hülle ein “Leben”. Es ist somit in der Lage, auf durch Benutzereingaben ausgelöste Ereignisse (Events) zu reagieren und wird zu einem handlungsoffenen Teil der virtuellen Welt.

2.2. Transformation statischer in interaktive 3D-Objekte

Die in der vorliegenden Arbeit untersuchte Transformation von statisch zu interaktiv ist keine rein geometrische oder visuelle Modifikation, sondern eine semantisch-verhaltensbezogene. Sie macht aus einem visuellen und geometrischen Asset ein interaktives Modell, welches Informationen über sein eigenes Verhalten und die möglichen Interaktionen mit diesem beinhaltet. Mit dem Konzept dieser "Smart Objects" wird Logik direkt im Objekt gespeichert, was wiederum Dezentralisierung und Wiederverwendbarkeit ermöglicht [1]. Dieser Prozess umfasst verschiedene Ebenen.

Zunächst wird das statische 3D-Modell segmentiert sowie semantisch etikettiert, was sich mit dem Begriff "Labeling" zusammenfassen lässt. Dabei werden rohe Mesh-Geometrien in bedeutungsvolle Teile mit bestimmten Rollen zerlegt (z. B. "Griff" einer Tasse oder "Sitzfläche" eines Stuhls), sodass ein System später "versteht", welches Teil eines 3D-Objekts für welche Interaktion relevant ist und eine Zuordnung stattfinden kann [6][7].

Im nächsten Schritt erfolgt der Übergang von der Semantik zum Verhalten mittels Zuweisung spezifischer Interaktionsmerkmale (Interaction Features) zu den im ersten Schritt analysierten semantischen Etiketten. Diese Merkmale beinhalten eine Beschreibung der Funktionalität sowie eine Verknüpfung mit Verhaltensplänen oder Zustandsautomaten. Diese Pläne können Zustände überprüfen oder ändern und legen das erwartete Verhalten von beteiligten Akteuren fest [1].

2.3. Das Vivian Framework

Das Vivian Framework ist ein Virtual-Prototyping-Framework. Es erstellt virtuelle Prototypen, also eine digitale, interaktive Repräsentation physischer Objekte aus der realen Welt. Diese Repräsentation kann z. B. in der Produktentwicklung verwendet werden, um virtuelle Prototypen von echten Produkten zu erzeugen und deren Verhalten frühzeitig zu testen, ohne dass das physische Produkt vorhanden sein muss. Diese virtuellen Prototypen funktionieren spezifikations-getrieben, sodass Verhalten nicht programmatisch, sondern deklarativ als Spezifikation geschrieben wird. Zur Laufzeit wird diese Spezifikation dann mittels einer State-Machine-basierten Umgebung interpretiert um ein Interaktionsverhalten abzubilden [8]. Ein Vorteil einer solchen spezifikations-getriebenen Vorgehensweise ist, dass beispielsweise Produktentwickler*innen keine Programmierkenntnisse benötigen, um virtuelle Prototypen von Produkten (Digital Twins) zu erstellen. So kann das Verhalten eines Produktes bereits frühzeitig und iterativ getestet werden. Ein weiteres mögliches Einsatzgebiet ist das Usability Engineering, mit dem interaktive Systeme so gestaltet und geprüft werden, dass sie für die jeweilige Zielgruppe möglichst effektiv, effizient sowie zufriedenstellend nutzbar sind. Dafür werden Systeme nicht erst am Ende, sondern bereits in der

Entwicklung, beispielsweise mittels Prototypen, auf Gebrauchstauglichkeit analysiert und getestet [9].

2.3.1. Spezifikationen

Wie bereits gezeigt, besitzen statische 3D-Objekte von sich aus keine Interaktivität (siehe 2.1). Das Vivian Framework bildet die Brücke zwischen statischem und interaktivem 3D-Objekt über eine maschinenlesbare Beschreibung des Verhaltens bei Interaktion mit dem virtuellen Prototyp. Die Verbindung dieser Spezifikation zu einem 3D-Objekt in der Szene wird über Namensreferenzen hergestellt. Ein Element in der Spezifikation referenziert demnach ein gleichnamiges 3D-Objekt oder Teilobjekt in der Szene. Weiterhin ermöglichen Spezifikationen eine Entkopplung des Verhaltensmodells vom 3D-Objekt, sodass unterschiedliche Spezifikationen verschiedene Verhalten bei demselben 3D-Objekt ermöglichen. Sie besteht im Wesentlichen aus 4 Teilen: Interaktionselemente (Interaction Elements), Visualisierungselemente (Visualization Elements), Zustände (States) und Übergänge (Transitions) [8].

Interaktionselemente. Hier befinden sich Elemente, welche Eingaben akzeptieren und somit Interaktion ermöglichen. Das können u. a. Buttons, Slider oder Movables sein.

Visualisierungselemente. Dabei handelt es sich Elemente, welche eine visuelle Rückmeldung geben. Typisch sind hier Light, Screen oder Sound Source.

Zustände. In diesem Teil befinden sich alle möglichen Zustände, die ein Prototyp erreichen kann. Jeder Zustand beschreibt eine bestimmte Situation, indem er Interaktions- und Visualisierungselementen Werte zuweist. Diese Zuweisung kann zudem an Bedingungen geknüpft sein.

Übergänge. Übergänge bestimmen, wie sich ein virtueller Prototyp von einem Zustand in einen anderen verändert. Diese Übergänge können durch Zeitüberschreitungen oder Ereignisse (Events) ausgelöst werden. Zusätzlich kann ein Übergang an eine zu erfüllende Bedingung (Guard) geknüpft sein.

2.4. Large Language Models und Agentische Systeme

2.4.1. Definition

Large Language Models (LLMs) sind tiefe neuronale Netzwerke, welche die Verarbeitung von natürlicher Sprache ermöglichen [10]. Sie enthalten typischerweise Milliarden von Parametern und sind in der Lage komplexe Aufgaben zu verstehen und zu lösen oder in Teilaufgaben zu zerlegen [11]. Ihre Skalierbarkeit ist ein zentrales Merkmal von LLMs, denn sie zeigen

mit zunehmender Modellgröße und Datenmenge qualitative Leistungssteigerungen und sogenannte emergente Fähigkeiten, wie z. B. *In-Context Learning*. Während solche LLMs als Modelle zur passiven Sprachverarbeitung und -generierung verstanden werden können, besitzen sie ohne zusätzliche Werkzeuge keine eigene Handlungsfähigkeit. Hier greift das Konzept eines LLM-basierten Agenten weiter. Ein Agent ist eine autonome Einheit, welche in einer Umgebung agiert, Entscheidungen trifft und auch mit anderen Agenten interagieren kann [12]. LLM-basierte Agenten verbinden diese klassische Idee mit den Fähigkeiten von LLMs. Ein LLM kann hier als kognitive Kernkomponente verstanden werden und verleiht dem Agent die Fähigkeit eigenständig und unabhängig zu analysieren, zu planen sowie Probleme zu lösen [13]. So entsteht ein System, welches nicht nur natürliche Sprache verarbeiten kann, sondern eine eigene zielgerichtete Handlungsfähigkeit besitzt und in einer Umgebung agieren kann. Ebenso kann es zusätzliche Komponenten wie Sensorik, API-Aufrufe oder Werkzeuge nutzen [13]. Die universelle Schnittstelle für solche agentischen Systeme, über die formuliert, geplant und koordiniert wird, ist die natürliche Sprache [14].

2.4.2. Multimodale LLMs

Multimodale Large Language Models (MLLMs) sind Modelle, welche im Gegensatz zu reinen LLMs nicht nur Text, sondern zudem Informationen wie Bild, Audio oder Video empfangen (Input), verarbeiten sowie ausgeben (Output) können. Eine für diese Arbeit wichtige Fähigkeit ist ihr Verstehen von 2D-Bildern und darüber zu schlussfolgern. Sie basieren auf LLMs und bestehen im Wesentlichen aus 3 Hauptmodulen: einem vortrainierten Encoder (Modality-Encoder), einem LLM sowie einem Adapter (Connector). Der Encoder verarbeitet die multimodalen Signale vor und der Adapter bildet die Brücke zum LLM, indem er diese Signale für das LLM verständlich umwandelt. Dieser Schritt ist nötig, da LLMs (wie in 2.4 gezeigt) ausschließlich Text verarbeiten können. Somit sind MLLMs in der Lage, Text sowie weitere multimodale Inhalte gemeinsam zu verstehen und darauf zu antworten [15], [16]. Im Vergleich zu früheren, ähnlichen Arbeiten basieren MLLMs auf LLMs mit Milliarden Parametern sowie neuen Trainingsparadigmen, wie dem “Multimodal Instruction Tuning” [17]. Damit sind eine Vielzahl von Möglichkeiten gegeben, welche mit reinen LLMs nicht oder nur eingeschränkt realisierbar wären. Solche repräsentativen Fähigkeiten sind u. a. das Verständnis von visuell geprägten Witzen/Memes [18], [19], räumliches/koordinatenbezogenes Verständnis, das Erstellen von Website-Code basierend auf handgezeichneten Entwürfen oder Kochrezepte anhand von Bildern zubereiteter Gerichte [19].

2.4.3. Verwendung in der vorliegenden Arbeit

In dieser Arbeit werden LLM-basierte Agenten als Steuerungseinheiten in einer Pipeline zur Spezifikationsgenerierung für das Vivian Framework verwendet. Sie übernehmen eine

zentrale Rolle in der Übersetzung von natürlicher Sprache in strukturierte, maschinenlesbare Spezifikationen. Sie analysieren 3D-Szenen, planen mögliche Interaktionen mit einem 3D-Objekt, Erzeugen Spezifikationen zur Realisierung dieser Interaktionen und validieren semantische Korrektheit der erzeugten Artefakte.

Kapitel 3.

Stand der Forschung

Die aktuelle Forschung in diesem Gebiet verbindet Sprachmodelle mit 3D-Repräsentationen und zeigt, dass LLM-basierte Systeme zunehmend in der Lage sind, 3D-Szenen nicht nur mit Sprache zu beschreiben, sondern semantisch sowie räumlich zu interpretieren. Mit der Weiterentwicklung von LLMs konnte deren Integration mit räumlichen Daten große Fortschritte erzielen und bietet so neue Möglichkeiten wie Szenenverständnis, Planung und Generierung von 3D-Welten oder Navigation in diesen. Spätestens seit 2023 hat sich die Verbindung zwischen LLMs und 3D-Repräsentationen zu einer eigenständigen Forschungslinie entwickelt, welche 3D-Szenen nicht nur beschreiben, sondern auch für neue Methoden wie das Grounding, Fragebeantwortung und Planungsaufgaben nutzen können [20], [21]. Trotz dieser schnellen Entwicklung bleiben Datenknappheit, Halluzinationsrobustheit und räumliches Verständnis (Grounding) zentrale Herausforderungen aktueller Systeme [2], [20].

3D-LLMs für 3D-Verstehen. In Arbeiten wie 3D-LLM und PointLLM können multimodale LLMs 3D-Daten direkt verarbeiten und nehmen dafür beispielsweise Punktwolken als Eingabe entgegen, wobei PointLLM eher objektzentriert ist. Dadurch können diese MLLMs relevante komplexere Konzepte wie räumliche Beziehungen zwischen Objekten, physikalische Eigenschaften, Raumlayouts oder Interaktionsmöglichkeiten (Affordances) verstehen [21], [22].

LLM-gestützte Pipelines zur Szenengenerierung und -bearbeitung. Parallel dazu wurden LLM-gestützte Pipelines entwickelt, in denen LLMs nicht direkt 3D-Daten verarbeiten. Stattdessen fungieren sie als Planungs- und Steuerungseinheit, die für die Generierung und Bearbeitung ganzer 3D-Szenen genutzt werden. Das Framework LLMR nutzt für die Generierung von interaktiven Szenen einen modularen Workflow, darunter die Module Planner, Scene Analyzer, Skill Library, Builder und Inspector. Es kann Szenenkontext analysieren und generiert anschließend Unity-kompatiblen C#-Code, welcher durch ein Inspektionsmodul iterativ geprüft wird. Als Eingabe arbeitet das System mit einer strukturierten, für das LLM konsumierbaren JSON-Repräsentation des Szenenkontexts [23]. HOLODECK verfolgt einen stärker szenenzentrierten Ansatz und zerlegt die Generierung einer Szene in einzelne

Schritte für Grundrisse, Materialien, Fenster und Türen, Objektauswahl sowie räumliche Anordnung. Es dient der automatisierten Erstellung von 3D-Simulationsumgebungen im Bereich der verkörperten KI (Embodied-AI) und platziert durch ein Constraint-basiertes Verfahren Objekte räumlich plausibel im Raum [24].

3.1. Artikulierung von statischen Objekte

Für die Transformation statischer in interaktive Objekte sind objekt- und szenenzentrierte Ansätze besonders relevant. ArtLLM wandelt eine Punktwolke in eine Abfolge von Tokens um, die vom Modell nacheinander generiert werden. Darüber hinaus können Assets auch als Bild oder Text eingegeben werden. Wichtig ist dabei, dass diese nicht direkt in das Kernmodell eingehen, sondern vorher mit Hilfe externer Generierungsmodelle in initiale Meshes und dann in Punktwolken umgewandelt werden. Diese Struktur definiert die Positionen der einzelnen Teile, die mechanischen Gelenke sowie deren Hierarchie. Um Positionen und Winkel anzugeben arbeitet es mit festen Rasterstufen statt Gleitkommazahlen. Nachdem detailliertere Geometrie über ein Zusatzmodul hinzugefügt wurde, wird diese kinematische Struktur als URDF-Datei (Unified Robotics Description Format) exportiert und kann so in Simulationen und virtuellen Umgebungen verwendet werden [25].

ArtiWorld erweitert diesen Ansatz von der Objekt- auf die Szenenebene. Das System nimmt eine textuelle Szenenbeschreibung entgegen und identifiziert auf dieser Grundlage artikulierbare Objekte. Anschließend erzeugt es strukturelle Beschreibungen der einzelnen Beziehungen und anschließend ebenfalls vollständige URDF-Dateien, welche an die ursprünglichen Koordinaten des starren 3D-Assets angepasst werden [26]. Die Geometrie des Originals wird dabei nicht ersetzt, sondern um Interaktivität erweitert. Insbesondere diese Arbeit zeigt, dass strukturelle Zwischenrepräsentationen das Reasoning verbessern.

3.2. Bildbasierte Eingaben

Eingaben basierend auf Bildern gewinnen in artikulationsbezogenen Pipelines an Bedeutung. Einzelbilder werden aktuell in mehreren Arbeiten genutzt, um simulierbare Szenen oder artikulierbare Objekte zu generieren, wie etwa in URDFormer oder ArtLLM [25], [27]. Dabei können Bilder sowohl isoliert, wie etwa in URDFormer, als auch zur ergänzenden Eingabe, wie in ArtLLM, verwendet werden, da diese semantische Hinweise und Informationen zu potenziellen Interaktionsmöglichkeiten bieten. Für die vorliegende Arbeit ist dieser Befund besonders relevant. Er zeigt die Kombination aus expliziten Raumdaten in Form von strukturellen Repräsentationen wie Punktwolken oder Szenengraphen mit Transformationen und Hierarchien einerseits sowie Bildern als visuellem Grounding andererseits.

3.3. Einordnung der vorliegenden Arbeit

In diesem Kontext ist die vorliegende Arbeit nicht primär als Verfahren zur Geometrie- oder Szenengenerierung einzuordnen, sondern als semantisch-verhaltensbezogener Ansatz zur Ableitung möglicher Interaktionen aus vorhandenen 3D-Szenen. Der wissenschaftliche Beitrag liegt hier somit in der multimodalen Analyse vorhandener 3D-Szenen und in der Überführung dieser in maschinenlesbare Spezifikationen für das Vivian Framework, mit welchen statische 3D-Objekte automatisiert interaktiv gemacht, und Geometrien wiederverwendet werden können.

Kapitel 4.

Methodik

In diesem Kapitel wird das methodische Vorgehen zur Entwicklung einer LLM-gestützten Pipeline zur Spezifikationsgenerierung für das Vivian Framework beschrieben. Ziel ist es ein System zu entwerfen, welches semantisch sowie strukturell korrekte Spezifikationen automatisiert erzeugt, Benutzer*innen-Feedback entgegennimmt sowie aufkommende Fehler iterativ und eigenständig behebt.

Diese Arbeit ist in Kooperation mit dem Institut für angewandte Informatik (IFAI) der Technischen Hochschule Nürnberg Georg-Simon-Ohm entstanden. Aufgrund dieser Zusammenarbeit sowie der Expertise im 2.3 beschriebenen Vivian Framework, wurde dieses als Grundlage zur Umsetzung einer solchen Pipeline genommen. Da dieses Framework interaktive Prototypen ausschließlich über eine maschinenlesbare Beschreibung des Verhaltens möglich macht, bietet es eine Grundlage, auf der LLM-gestützte Verfahren aufsetzen können.

4.1. Forschungs- und Entwicklungsansatz

4.2. Anforderungsanalyse

Im Folgenden wird die LLM-gestützte Pipeline zur Generierung von Vivian Spezifikationen *Vivian Specification Generator (VSG)* genannt.

4.2.1. Funktionale Anforderungen

A1: Der VSG muss als modulare Verarbeitungsfolge mit klar definierten Ein- und Ausgaben pro Modul umgesetzt sein.

A2: Der VSG erzeugt aus bestehenden Szenendaten und natürlichsprachlichen Beschreibungen Vivian-kompatible Spezifikationsdateien.

A3: Vorhandene Geometrien und Hierarchien werden vom VSG nicht neu generiert, sondern ausschließlich mit Semantik und Verhalten angereichert.

A4: Relevante Objekte, Teilobjekte und Rollen müssen explizit identifiziert und referenzierbar gemacht werden.

A5: Um Fehler frühzeitig abzufangen, führt der VSG syntaktische, semantische und referenzielle Prüfungen durch und korrigiert erkannte Fehler iterativ bis zu einer definierten Grenze.

A6: Um Szeneninterpretationen überprüfen und gegebenenfalls korrigieren zu können, soll ein Human-in-the-Loop-Schritt implementiert werden. Dieser nimmt gewünschte natürlichsprachlich beschriebene Korrekturen entgegen und interpretiert die Szene damit erneut.

4.2.2. Nicht-funktionale Anforderungen

N1: Zwischenartefakte und Entscheidungen des VSG werden protokolliert und abgespeichert.

N2: Der VSG soll modular aufgebaut sein.

N3: Erkannte Fehler aus der Validierung sollen für Korrekturschritte nutzbar sein.

N4: Die Implementierung des VSG soll prototypisch erweiterbar bleiben.

4.2.3. Domänenspezifische Anforderungen

D1: Die Ausgabe der Pipeline muss den von Vivian erwarteten Spezifikationsdateien entsprechen.

D2: Interaktionselemente, Visualisierungselemente, Zustände und Übergänge müssen konsistent benannt sein.

D3: Übergänge müssen auf vorhandene Elemente und Zustände referenzieren.

D4: Die generierte Spezifikation muss in Unity mit Vivian ausführbar sein.

4.3. Spezifikationsgenerierung für das Vivian Framework

4.4. Konzeption der Pipeline

4.4.1. Scene Analyzer

4.4.2. Interaction Planner

4.4.3. Specification Generator

4.4.4. Consistency Reviewer

4.4.5. Validator

Kapitel 5.

Implementierung

Kapitel 6.

Evaluation

Kapitel 7.

Ausblick

Kapitel 8.

Fazit

Anhang A.

Supplemental Information

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Das hier ist der zweite Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Und nun folgt – ob man es glaubt oder nicht – der dritte Absatz. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Nach diesem vierten Absatz beginnen wir eine neue Zählung. Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln.

Abbildungsverzeichnis

Tabellenverzeichnis

Listingverzeichnis

Literatur

[1] M. Kallmann und D. Thalmann, „Modeling Behaviors of Interactive Objects for Real-Time Virtual Environments,“ *Journal of Visual Languages & Computing*, Jg. 13, Nr. 2, S. 177–195, Apr. 2002, ISSN: 1045926X. DOI: 10.1006/jvlc.2001.0229 besucht am 4. Apr. 2026.

[2] R. Peng, K. Li, W. Zhang, C. Gao, X. Chen und Y. Li, *Understanding and Evaluating Hallucinations in 3D Visual Language Models*, Feb. 2025. DOI: 10.48550/arXiv.2502.15888 arXiv: 2502.15888 [cs]. besucht am 18. Apr. 2026.

[3] I. Baran und J. Popović, „Automatic Rigging and Animation of 3D Characters,“ *ACM Trans. Graph.*, Jg. 26, Nr. 3, 72–es, Juli 2007, ISSN: 0730-0301. DOI: 10.1145/1276377.1276467 besucht am 8. Apr. 2026.

[4] *Gabriel Zachmann's Dissertation*, https://user.informatik.uni-bremen.de/zach/papers/diss.html. besucht am 9. Apr. 2026.

[5] J. Steuer, „Defining Virtual Reality: Dimensions Determining Telepresence,“ *Journal of Communication*, Jg. 42, Nr. 4, S. 73–93, 1992, ISSN: 1460-2466. DOI: 10.1111/j.1460-2466.1992.tb00812.x besucht am 9. Apr. 2026.

[6] K. Mo u. a., *PartNet: A Large-scale Benchmark for Fine-grained and Hierarchical Part-level 3D Object Understanding*, Dez. 2018. DOI: 10.48550/arXiv.1812.02713 arXiv: 1812.02713 [cs]. besucht am 4. Apr. 2026.

[7] M. Hassanin, S. Khan und M. Tahtali, „Visual Affordance and Function Understanding: A Survey,“ *ACM Comput. Surv.*, Jg. 54, Nr. 3, 47:1–47:35, Apr. 2021, ISSN: 0360-0300. DOI: 10.1145/3446370 besucht am 4. Apr. 2026.

[8] *Vivian Framework · GitLab*, https://gitlab.com/usability-engineering-ar-mr-vr/vivian-framework, Mai 2021. besucht am 16. Apr. 2026.

[9] J. D. Gould und C. Lewis, „Designing for Usability: Key Principles and What Designers Think,“ *Commun. ACM*, Jg. 28, Nr. 3, S. 300–311, März 1985, ISSN: 0001-0782. DOI: 10.1145/3166.3170 besucht am 11. Apr. 2026.

[10] S. J. Lazer, K. Aryal, M. Gupta und E. Bertino, *A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes*, Jan. 2026. DOI: 10.48550/arXiv.2601.05293 arXiv: 2601.05293 [cs]. besucht am 10. Apr. 2026.

[11] R. Bommasani u. a., *On the Opportunities and Risks of Foundation Models*, Juli 2022. DOI: 10.48550/arXiv.2108.07258 arXiv: 2108.07258 [cs]. besucht am 9. Apr. 2026.

[12] Y. Shoham, „Agent-Oriented Programming," *Artificial Intelligence*, Jg. 60, Nr. 1, S. 51–92, März 1993, ISSN: 0004-3702. DOI: 10.1016/0004-3702(93)90034-9 besucht am 8. Apr. 2026.

[13] Z. Xi u. a., *The Rise and Potential of Large Language Model Based Agents: A Survey*, Sep. 2023. DOI: 10.48550/arXiv.2309.07864 arXiv: 2309.07864 [cs]. besucht am 8. Apr. 2026.

[14] Y. Shen, K. Song, X. Tan, D. Li, W. Lu und Y. Zhuang, *HuggingGPT: Solving AI Tasks with ChatGPT and Its Friends in Hugging Face*, Dez. 2023. DOI: 10.48550/arXiv.2303.17580 arXiv: 2303.17580 [cs]. besucht am 8. Apr. 2026.

[15] S. Yin u. a., „A Survey on Multimodal Large Language Models," *National Science Review*, Jg. 11, Nr. 12, nwae403, Dez. 2024, ISSN: 2095-5138. DOI: 10.1093/nsr/nwae403 besucht am 13. Apr. 2026.

[16] D. Caffagni u. a., „The Revolution of Multimodal Large Language Models: A Survey," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins und V. Srikumar, Hrsg., Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, S. 13 590–13 618. DOI: 10.18653/v1/2024.findings-acl.807 besucht am 13. Apr. 2026.

[17] H. Liu, C. Li, Q. Wu und Y. J. Lee, „Visual Instruction Tuning,"

[18] Z. Yang u. a., *MM-REACT: Prompting ChatGPT for Multimodal Reasoning and Action*, März 2023. DOI: 10.48550/arXiv.2303.11381 arXiv: 2303.11381 [cs]. besucht am 13. Apr. 2026.

[19] D. Zhu, J. Chen, X. Shen, X. Li und M. Elhoseiny, *MiniGPT-4: Enhancing Vision-Language Understanding with Advanced Large Language Models*, Okt. 2023. DOI: 10.48550/arXiv.2304.10592 arXiv: 2304.10592 [cs]. besucht am 13. Apr. 2026.

[20] X. Ma u. a., *When LLMs Step into the 3D World: A Survey and Meta-Analysis of 3D Tasks via Multi-modal Large Language Models*, Okt. 2025. DOI: 10.48550/arXiv.2405.10255 arXiv: 2405.10255 [cs]. besucht am 3. Apr. 2026.

[21] Y. Hong, H. Zhen, P. Chen und S. Zheng, „3D-LLM: Injecting the 3D World into Large Language Models,"

[22] R. Xu, X. Wang, T. Wang, Y. Chen, J. Pang und D. Lin, „PointLLM: Empowering Large Language Models to Understand Point Clouds," in *Computer Vision – ECCV 2024*, A. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler und G. Varol, Hrsg., Bd. 15083, Cham: Springer Nature Switzerland, 2025, S. 131–147, ISBN: 978-3-031-72697-2 978-3-031-72698-9. DOI: 10.1007/978-3-031-72698-9_8 besucht am 18. Apr. 2026.

[23] F. De La Torre, C. M. Fang, H. Huang, A. Banburski-Fahey, J. Amores Fernandez und J. Lanier, „LLMR: Real-time Prompting of Interactive Worlds using Large Language Models,“ in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, Honolulu HI USA: ACM, Mai 2024, S. 1–22, ISBN: 979-8-4007-0330-0. DOI: 10.1145/3613904.3642579 besucht am 16. Apr. 2026.

[24] Y. Yang u. a., *Holodeck: Language Guided Generation of 3D Embodied AI Environments*, Apr. 2024. DOI: 10.48550/arXiv.2312.09067 arXiv: 2312.09067 [cs]. besucht am 15. Apr. 2026.

[25] P. Wang u. a., *ArtLLM: Generating Articulated Assets via 3D LLM*, März 2026. DOI: 10.48550/arXiv.2603.01142 arXiv: 2603.01142 [cs]. besucht am 16. Apr. 2026.

[26] Y. Yang u. a., *ArtiWorld: LLM-Driven Articulation of 3D Objects in Scenes*, Nov. 2025. DOI: 10.48550/arXiv.2511.12977 arXiv: 2511.12977 [cs]. besucht am 20. Apr. 2026.

[27] Z. Chen u. a., *URDFormer: A Pipeline for Constructing Articulated Simulation Environments from Real-World Images*, Mai 2024. DOI: 10.48550/arXiv.2405.11656 arXiv: 2405.11656 [cs]. besucht am 20. Apr. 2026.